

מבוא:

מטרת תמצית הקורס היא לסייע לכם ללמוד את החומר שנלמד בקורס ולארגן אותו בצורה מסודרת, כך שיהיה נגיש ומרוכז. חשוב להדגיש כי תמצית הקורס אינה מהווה תחליף לחזרה על החומר שהועבר בכיתה ואינה מכילה את מלוא הידע הדרוש להצלחה בבחינה. הדרך הנכונה להשתמש בתמצית הינה להשתתף באופן פעיל בהרצאות ובמעבדות, לחזור על מצגות ההרצאה והמעבדה ורק אז להעזר בתמצית כדי להבין את התמונה הגדולה.

טבלה 1: קשרים בין מחלקות	2
טבלה 2: עקרונות עיצוב מחלקות SOLID	3
טבלה 3: עקרונות עיצוב חבילות	4
טבלה 4: מדדי עיצוב חבילות	5
טבלה 5: סיכום תבניות עיצוב	6
טבלה 6: תרשימי תבניות עיצוב	8
טבלה 7: סיכום פונקציות Stream	13

תחנת לניטור מזג אוויר	14
עיצוב ראשוני:	14
בעיות עיצוב:	14
שיפורים:	14
הוספת חיישן למדידת מגמת לחץ האוויר	16
תבנית עיצוב Bridge	16
שימוש ב Bridge עבור השעון	17
אתחול	17
תבנית עיצוב Proxy	18
חלוקה לחבילות:	18
מקרה בוחן טלפון סלולרי	19
תרשימי מצבים	20
תרשים מכונת כביסה	20

שם	דיאגרמה	משמעות
Dependency		A מכיר את B. דוגמאות: A יוצר B, A מקבל את B כפרמטר לפונקציה/לבנאי יש ב A Include B.
Directed Association		מחלקה A משתמשת במחלקה B. A uses a B. ל A יש מצביע מסוג B.
Bidirectional Association		יש שימוש הדדי בין A ל B. בכל פעם שמחברים/מנתקים בין מופע של A למופע של B, החיבור/ניתוק חל גם בכיוון השני.
Directed Aggregation		למחלקה A יש מחלקה B, A has a B. כמו Association, בתוספת בעלות. לכל אובייקט b מסוג B, לכל היותר לאובייקט אחד מסוג A יש מצביע itsB לאובייקט b.
Bidirectional Aggregation		כמו Directed Aggregation. B מכיל מצביע חזרה ל A. בכל פעם שמחברים/מנתקים בין מופע של A למופע של B, החיבור/ניתוק חל גם בכיוון השני.
Directed Composition		A מורכב מ B. יחס של שלם-חלק. כשנאתחל/נמחק אובייקט מסוג A, יאותחל/ימחק גם ה itsB שלו.
Bidirectional Composition		כמו Directed Composition. B מכיל מצביע חזרה ל A. בכל פעם שמחברים/מנתקים בין מופע של A למופע של B, החיבור/ניתוק חל גם בכיוון השני.
Composite		משמעות זהה ל Composition. לא ניתן לדעת מהתרשים אם הקשר הוא חד/דו כיווני.
Link		מופיע בתוך Composite. משתמע כי ב A יש מצביע מסוג B. כאשר ה Composite מאותחל, יאותחלו גם itsA itsB itsA יצביע ל itsB.
Realization		A מממש את ממשק B
Generalization		A יורש ממחלקה B

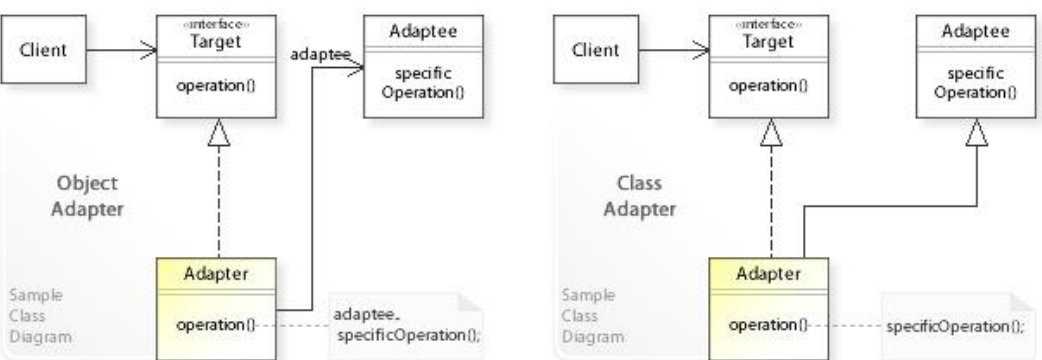
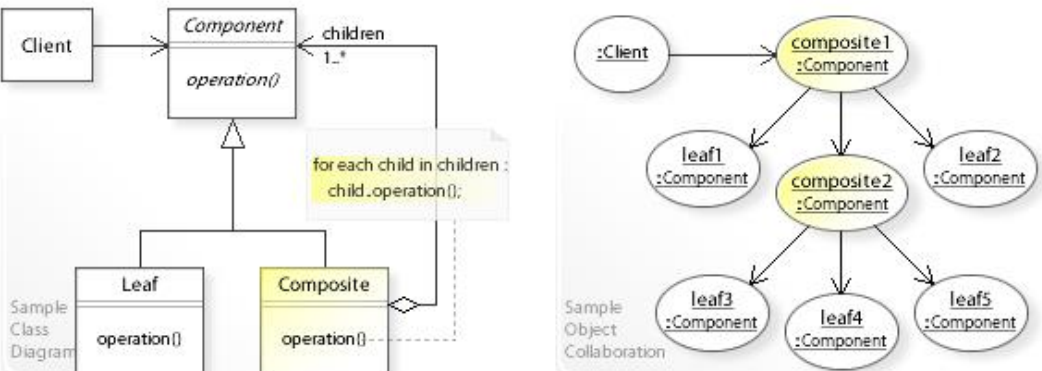
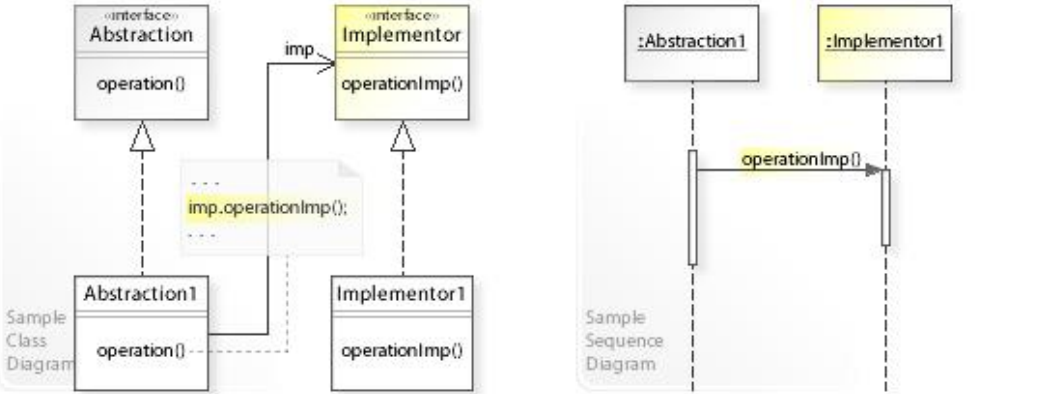
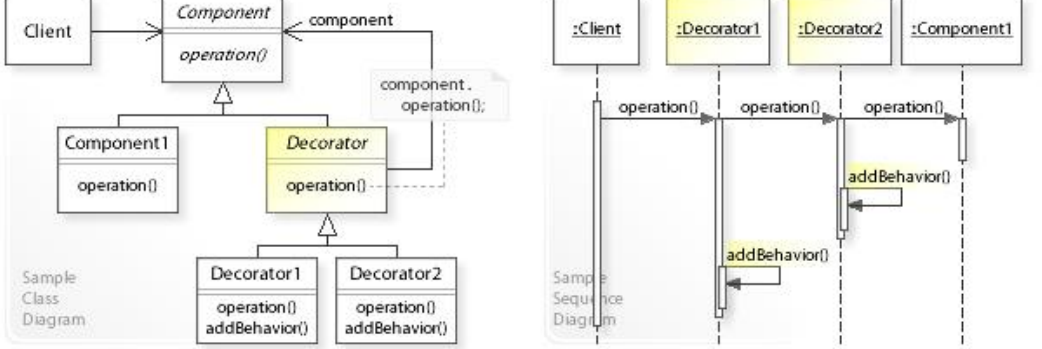
קיצור	שם מלא של העקרון	הגדרת העיקרון	פתרון הפרה
OCP	Open-Closed Principle	תוכנה צריכה להיות פתוחה להרחבה וסגורה לשינויים (אפילו לקומפילציה מחדש)	שימוש בהפשטה. קיום שאר העקרונות
SRP	Single Responsibility Principle	כל מודול/מחלקה אמורה להיות אחראית לחלק יחיד מהפונקציונליות שהתכנה מספקת. אחריות זו אמורה להיות מוכמסת (encapsulated) על ידי המחלקה, וכל שירותי המחלקה אמורים להיות מותאמים לאחריות יחידה זו. לכל מחלקה אמורה להיות סיבה יחידה להשתנות.	פיצול מחלקה למחלקות עם אחריות נפרדת. העברת פונקציות ממחלקה אחת למחלקה שאחראית לפונקציה.
DIP	Dependency Inversion Principle	מודול בשכבה מסויימת לא יהיה תלוי ישירות בשכבה נמוכה יותר, התלות ביניהם תהיה באמצעות הפשטה. מודולים מופשטים לא צריכים להיות תלוי בפרטים. הפרטים צריכים להיות תלויים במודולים מופשטים.	הוספת ממשק שהמחלקה שתלויים בה מממשת. הקליינטים יהיו תלויים בממשק.
LSP	Liskov Substitution Principle	תת-טיפוסים חייבים להיות ברי החלפה (התנהגותית) עבור טיפוס הבסיס שלהם. קליינט המשתמש ב-pointer או ב-reference למחלקת בסיס, לא צריך לדעת לאיזה תת-מחלקה האובייקט שייך. ההתנהגות של הבנים צריכה להיות דומה לשל האבא.	לא להשתמש בירושה במקרים בהם העקרון לא מתקיים. עדכון החוזה על פיו מוגדרת מחלקת האב, כך שהמשתמש לא יופתע מהתנהגות הבנים.
ISP	Interface Segregation Principle	הקליינט יהיה חשוף אך ורק לממשק שהוא זקוק לו אסור להכריח קליינט להיות תלוי בממשק בו הוא לא משתמש	פיצול ממשק למספר ממשקים.

קיצור	שם מלא של העקרון	סוג	הגדרת העיקרון המקורית	פרשנות	פתרון הפרה
REP	Reuse/Release Equivalence Principle	לכידות	יחידת השימוש החוזר בקוד היא גם יחידת שחרור הקוד. רק רכיבים שמשוחררים דרך מערכת לניהול גרסאות יכולות לעבור שימוש חוזר. היחידה הזו היא החבילה.	חבילת תכנה הוא היחידה הקטנה ביותר תחת ניהול תצורה. אסור לשחרר רק חלק מחבילה. אסור להשתמש מחדש רק בחלק מחבילה.	תהליך מסודר של שחרור/שימוש חוזר בחבילות
CRP	Common Reuse Principle	לכידות	המחלקות בחבילה עוברות שימוש חוזר יחד. אם משתמשים במחלקה בחבילה מסויימת, משתמשים בכולן.	משתמשים בכל המחלקות של החבילה ולא רק בחלק מהם. לא נרצה להידרש לעדכן גרסה כשרק חלק ממנה דרוש לנו. הקליינט לא אמור להיות תלוי במחלקות שהוא לא משתמש בהן	פיצול החבילה בצורה שכל קליינט משתמש בחבילה שלמה בלבד.
CCP	Common Closure Principle	לכידות	המחלקות בחבילה צריכות להיות סגורות יחד כנגד אותו סוג של שינויים. שינוי שמשפיע על חבילה, משפיע על כל המחלקות השייכות לחבילה.	צריך לקבץ באותה חבילה מחלקות המושפעות מאותו סוג של שינויים. לחבילה צריכה להיות רק סיבה אחת להשתנות	איחוד חבילות, כך שמחלקות שמושפעות מאותו סוג שינוי יימצאו באותה חבילה.
ADP	Acyclic Dependencies Principle	צימוד	מבנה התלויות בין החבילות חייב להיות גרף מכוון חסר מעגלים.	אסור שיהיה מעגל של תלויות בין חבילות.	שימוש בטכניקת DIP לשבירת המעגל.
SDP	Stable Dependencies Principle	צימוד	התלויות בין חבילות צריכות להיות בכיוון היציבות של החבילות. חבילה צריכה להיות תלויה אך ורק על חבילות היציבות יותר ממנה.	מדד ה'א' (האי-יציבות) של חבילה צריך להיות (שווה או) גבוה יותר מזה של החבילות שהוא תלוי בהן.	ארגון מחדש של החבילות, הוספת חבילות, הפיכת התלות בין חבילות.
SAP	Stable Abstractions Principle	צימוד	חבילות יציבות לחלוטין צריכות להיות מופשטות לחלוטין. חבילות לא יציבות צריכות להיות קונקרטיות. רמת המופשטות של חבילה צריכה להיות ביחס ישר ליציבות שלה.	אם מדד ה'א' של חבילה כלשהי הוא 0, אז מדד ה-A שלה צריך להיות 1. מדד ה-A (המופשטות) של חבילה צריך להיות נמוך יותר מזה של החבילות שהוא תלוי בהן. בהינתן סף שנקבע על ידי צוות הפיתוח, לכל החבילות אמור להיות D' מתחת לסף.	ארגון מחדש של החבילות, הוספת חבילות, הפיכת התלות בין חבילות.

שם הממד	הגדרה
עצמאות	מדד איכותני. חבילה עצמאית יותר ככל שהיא תלויה בפחות מחלקות חיצוניות. חבילה עצמאית היא חבילה שאינה תלויה במחלקות חיצוניות.
אחריות	מדד איכותני. חבילה אחראית יותר ככל שיש יותר מחלקות חיצוניות התלויות בה. חבילה אחראית היא חבילה שלפחות מחלקה חיצונית אחת תלויה בה.
Ca	מספר המחלקות בחבילות אחרות, התלויות במחלקות בתוך החבילה (תלויות נכנסות).
Ce	מספר המחלקות בחבילות אחרות, שמחלקות בתוך ה Package תלויות בהן (תלויות יוצאות)
S: יציבות	$S = \frac{Ca}{(Ca+Ce)}$
A: מפשטות	$A = \frac{\#Abstract\ classes}{\#Classes}$
D: מרחק מהקו המנחה	$D = \left \frac{A - S}{\sqrt{2}} \right $
D': מרחק מנורמל	$D' = A - S $

שם התבנית	סוג	תיאור הבעיה/מטרה	וריאנטים	השלכות	מקרי בוחן
Adapter	מבנה	1. מעוניינים לשלב בעיצוב מחלקה קיימת שאינה מממשת את הממשק שאנו משתמשים בו. 2. מעוניינים לאפשר למחלקה אחת שני מימושים שונים של אותו ממשק	Class Adapter	המתאם הוא סוג של המתואם.	Button-Lamp Cellphone
			Object Adapter	העדפת הכלה על ירושה. דורש אובייקט נפרד.	Timed security door Weather Monitoring System
Composite	מבנה	האובייקטים מאורגנים במבנה היררכי (עץ). נרצה שהלקוחות יוכלו להתעלם מההבדלים בין הרכבה של אובייקטים לבין אובייקטים בודדים, וייתייחסו לשניהם באופן אחיד.	Composite מנוון המאפשר רק רמת היררכיה אחת		Hierarchical statecharts Report Generator (Visitor)
Bridge	מבנה	הפרדה מוחלטת בין שכבות הפשטה שונות. התבנית מאפשרת לכל שכבה להשתנות באופן עצמאי ומנתקת את התלות של שכבה אחת בשכבה השניה.			Weather Monitoring System (Sensors/Clock)
Decorator	מבנה	שכלול פעולות קיימות במחלקה בדרכים חדשות ובלתי תלויות, הניתנות לשילוב ביניהן.	Simple Decorator	אם מקשטים יותר מפונקציה אחת, דורש שכפול קוד	Modem, Sukkah
			Multiple Decorators		
Flyweight	מבנה	לחלוק ביעילות קבוצה גדולה של אובייקטים קטנים.			Car-Sales
Proxy	מבנה	מאפשר לשלוט בגישה לאובייקט. משמש כדי להוסיף בקרה על השימוש באובייקט "יקר" או "מוגן". משמש להפרדה בין Persistence layer לבין Policy layer.			Weather Monitoring System
Singleton	יצירה	הגבלה של יצירת המופעים של מחלקה מסוימת למופע יחיד.	Thread-Safe	יכול להוביל לפגיעה בביצועים	Weather Monitoring System
			Eager	פותר בעיית מקבול יכול לבזבז משאבים	
			Multi-Singleton		
Abstract Factory	יצירה	יצירת אובייקטים באופן גמיש. יצירת משפחה של אובייקטים שהטיפוסים שלהם אמורים להיות מתואמים.			Weather Monitoring System Dishwasher

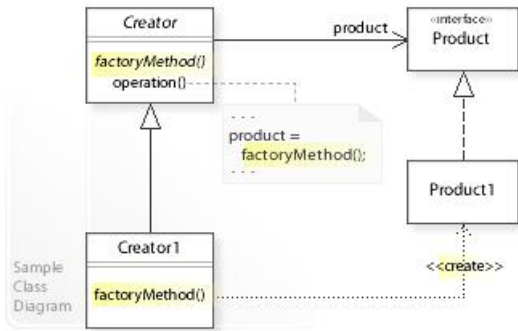
שם התבנית	סוג	תיאור הבעיה/מטרה	וריאנטים	השלכות	מקרי בוחן
Factory method	יצירה	מחלקה המגדירה ממשק לייצור אובייקט. המחלקה היורשת תקבע את הסוג הקונקרטי של האובייקט.			
Factory Class		מחלקה שבה יש פונקציה לייצור אובייקטים			
Prototype	יצירה	יצירת אובייקטים חדשים על ידי שכפול אובייקטים קיימים בעלי יכולת שיבוט (Cloning) וללא צורך להכיר את הטיפוס של האובייקט המיוצר.			Car-Sales
Observer	התנהגות	אובייקט אחד יכול לעדכן מספר כלשהו של אובייקטים אחרים בכל פעם שהמצב שלו משתנה בלי שהעיצוב יכלול צימוד הדוק.			Weather Monitoring System
Iterator	התנהגות	מעוניינים לעבור על איברי אוסף, אך לנתק את התלות בין האוסף הקונקרטי לאלגוריתם			Security door Weather Monitoring System
Memento	התנהגות	לבטל את הפעולה האחרונה שבוצעה (Undo) ולחזור למצב קודם.			
Strategy	התנהגות	הגדרת משפחה של אלגוריתמים ברי החלפה שכל אחד מהם יכול להתפתח ללא תלות בלקוחות שלו.			
Command	התנהגות	כימוס פעולה אחת בצורת אובייקט			Remote control
Visitor	התנהגות	הגדרה של פעולה חדשה למחלקות ממשפחה מסוימת (מעבר לאחריות של המחלקות) בלי לשנות את המחלקות עצמן.	Simple Visitor Acyclic Visitor		Modem Report Generator
Template method	התנהגות	הכנסת "וויים" (hooks) לתוך הקוד של מתודה שמאפשרת הרחבה של ההתנהגות שלה, תוך שמירה על המבנה הבסיסי שלה.			Weather Monitoring System
State	התנהגות	התנהגות האובייקט תלויה במצבו, והוא חייב לשנותה בזמן ריצה (בתלות במצבו הפנימי). הוספת מצבים חדשים לא אמורה לשנות את ההתנהגות של מצבים קיימים.			Dishwasher

תרשים	שם התבנית
 Object Adapter Sample Class Diagram <pre> classDiagram class Client class Target { <<interface>> operation() } class Adapter { operation() } class Adaptee { specificOperation() } Client --> Target Adapter .. > Target Adapter --> Adaptee : adaptee Adapter --> Target : operation() Adapter ..> Adaptee : adaptee.specificOperation(); </pre> Class Adapter Sample Class Diagram <pre> classDiagram class Client class Target { <<interface>> operation() } class Adapter { operation() } class Adaptee { specificOperation() } Client --> Target Adapter .. > Target Adapter --> Adaptee Adapter --> Target : operation() Adapter ..> Adaptee : specificOperation(); </pre>	Adapter
 Sample Class Diagram <pre> classDiagram class Client class Component { <<interface>> operation() } class Leaf { operation() } class Composite { operation() } Client --> Component Component < .. Leaf Component < .. Composite Composite --> Component : children 1..* Composite --> Component : for each child in children : child.operation(); </pre> Sample Object Collaboration <pre> sequenceDiagram participant Client as :Client participant composite1 as composite1 :Component participant leaf1 as leaf1 :Component participant composite2 as composite2 :Component participant leaf2 as leaf2 :Component participant leaf3 as leaf3 :Component participant leaf4 as leaf4 :Component participant leaf5 as leaf5 :Component Client->>composite1: activate composite1 composite1->>leaf1: deactivate leaf1 composite1->>composite2: activate composite2 composite2->>leaf3: deactivate leaf3 composite2->>leaf4: deactivate leaf4 composite2->>leaf5: deactivate leaf5 deactivate composite2 deactivate composite1 </pre>	Composite
 Sample Class Diagram <pre> classDiagram class Abstraction { <<interface>> operation() } class Implementor1 { <<interface>> operationImp() } class Abstraction1 { operation() } class Implementor1_1 { operationImp() } Abstraction < .. Abstraction1 Implementor1 < .. Implementor1_1 Abstraction1 --> Implementor1_1 : imp Abstraction1 --> Abstraction : operation() Abstraction1 ..> Implementor1_1 : imp.operationImp(); </pre> Sample Sequence Diagram <pre> sequenceDiagram participant Abstraction1 as :Abstraction1 participant Implementor1 as :Implementor1 Abstraction1->>Implementor1: operationImp() activate Implementor1 Implementor1-->>Abstraction1: deactivate Implementor1 </pre>	Bridge
 Sample Class Diagram <pre> classDiagram class Client class Component { <<interface>> operation() } class Component1 { operation() } class Decorator { operation() } class Decorator1 { operation() addBehavior() } class Decorator2 { operation() addBehavior() } Client --> Component Component < .. Component1 Component < .. Decorator Decorator --> Component : component Decorator --> Component : component.operation(); Decorator1 < .. Decorator Decorator2 < .. Decorator </pre> Sample Sequence Diagram <pre> sequenceDiagram participant Client as :Client participant decorator1 as :Decorator1 participant decorator2 as :Decorator2 participant component1 as :Component1 Client->>decorator1: operation() activate decorator1 decorator1->>decorator2: operation() activate decorator2 decorator2->>component1: operation() activate component1 component1-->>decorator2: deactivate component1 decorator2->>decorator1: addBehavior() deactivate decorator2 decorator1->>decorator1: addBehavior() deactivate decorator1 </pre>	Decorator

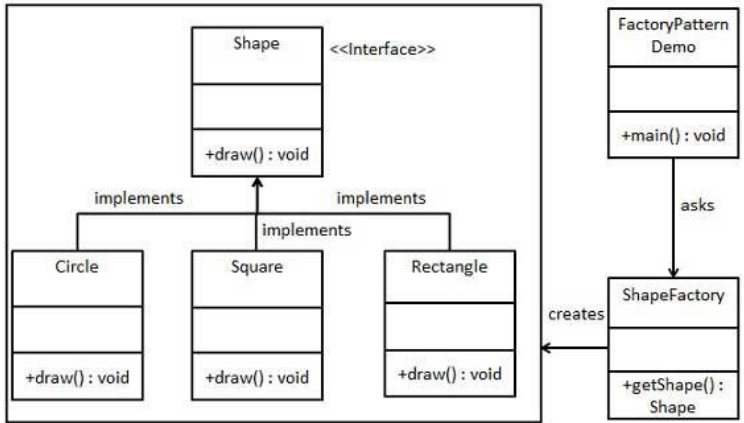
תרשים		שם התבנית	
Sample Class Diagram Sample Sequence Diagram	Flyweight		
Sample Class Diagram Sample Sequence Diagram	Proxy		
Singleton - singleton : Singleton - Singleton() + getInstance() : Singleton		Singleton	
Sample Class Diagram Sample Sequence Diagram	Abstract Factory		

תרשים

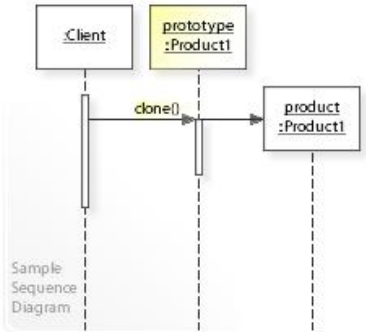
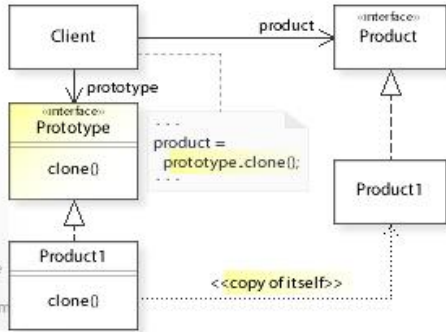
שם התבנית



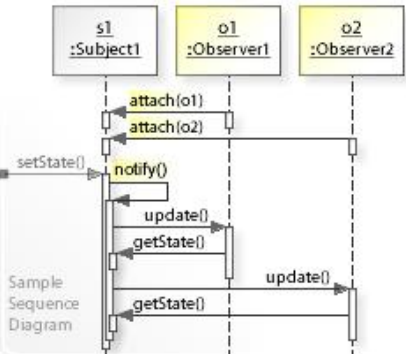
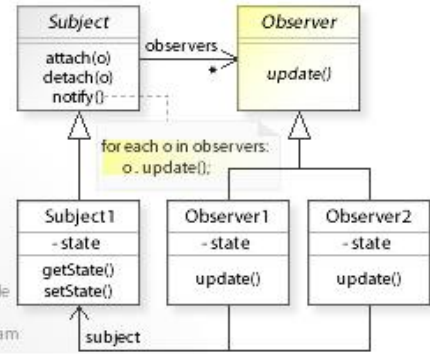
Factory
method



Factory
Class



Prototype

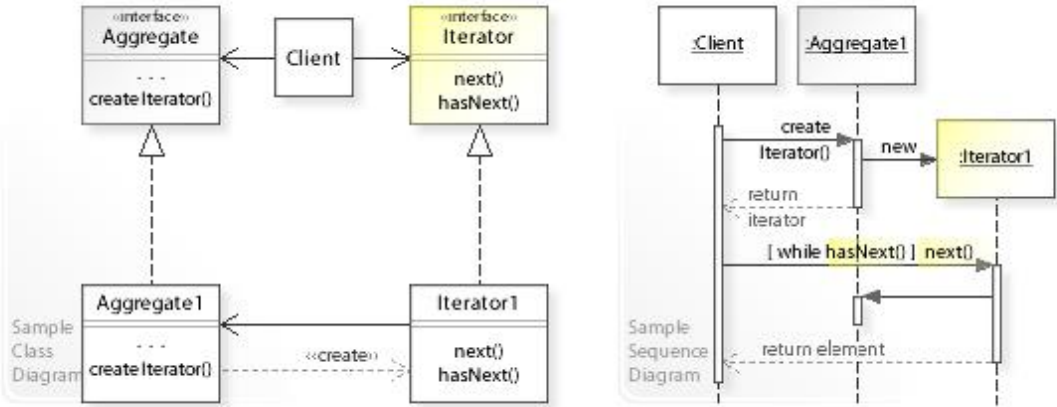


Observer

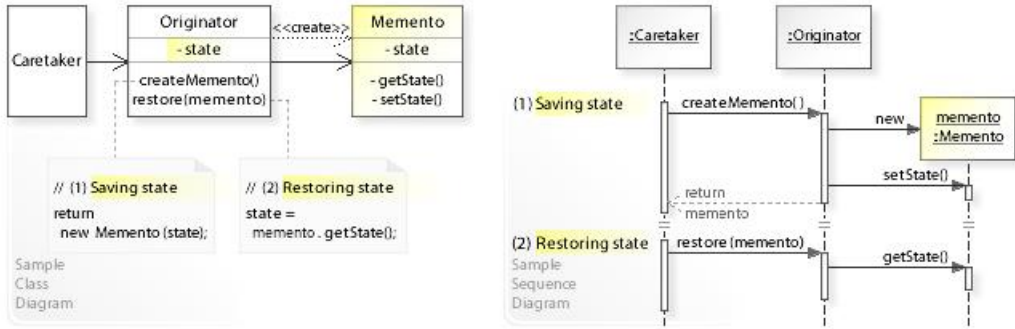
תרשים

שם התבנית

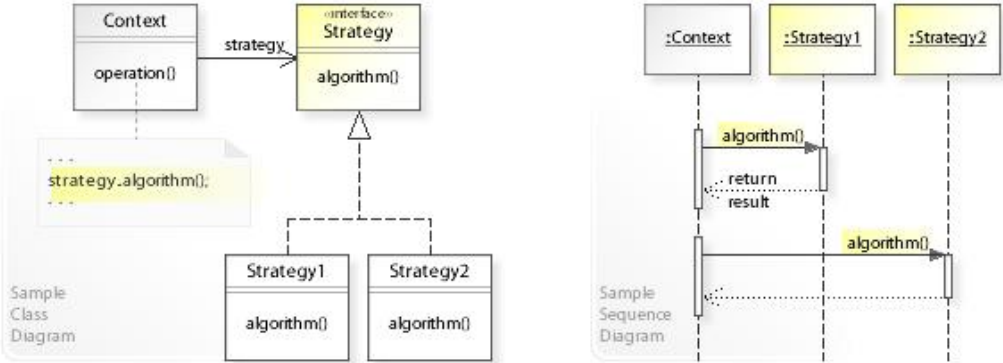
Iterator



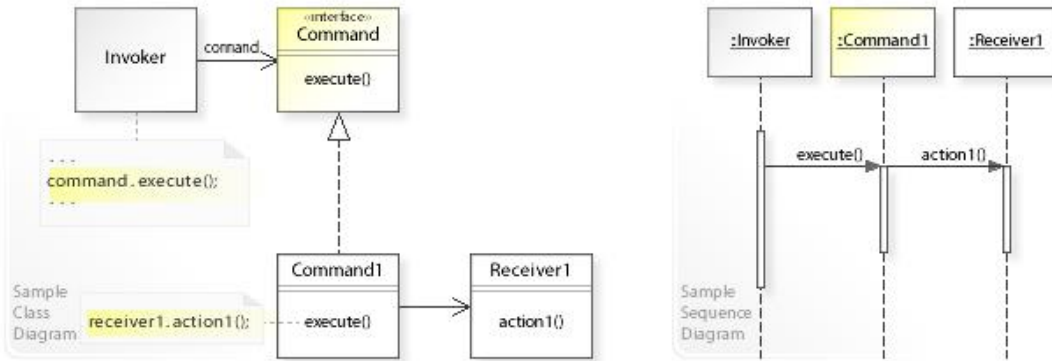
Memento



Strategy



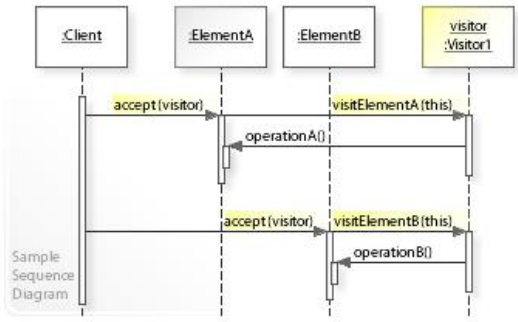
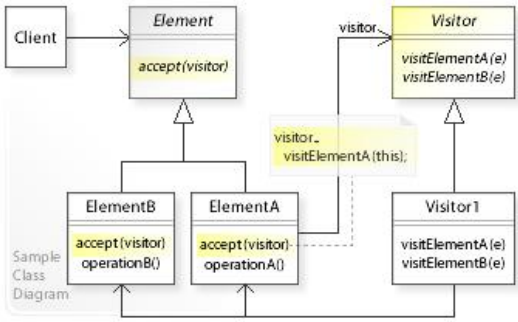
Command



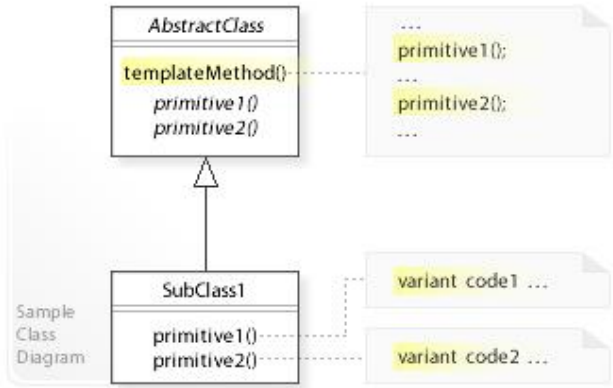
תרשים

שם התבנית

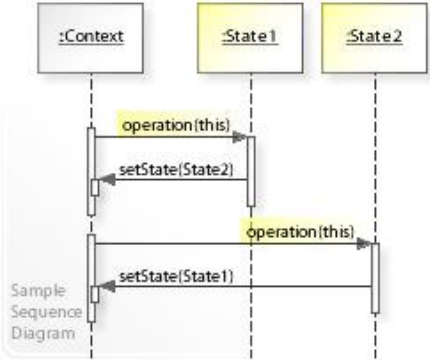
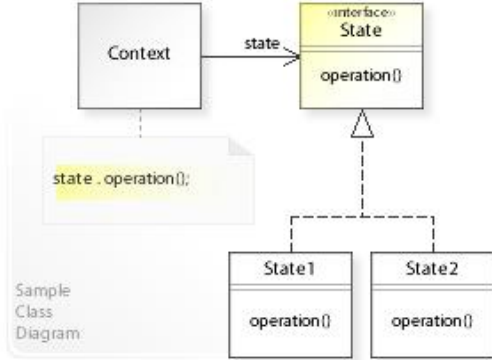
Visitor



Template method

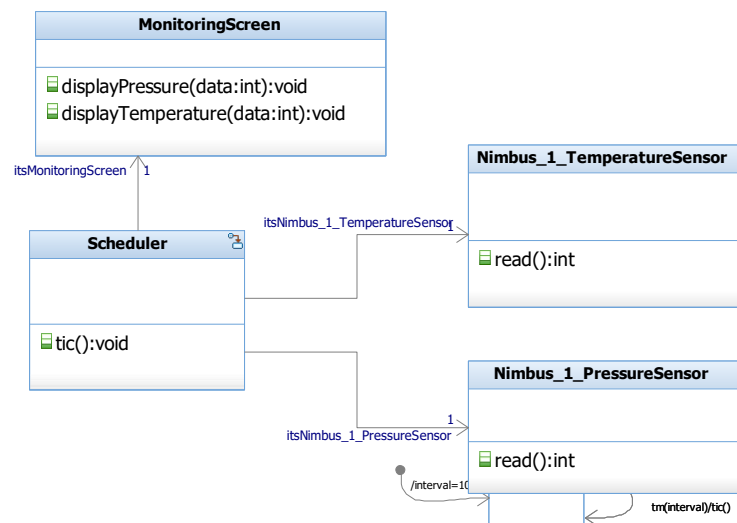
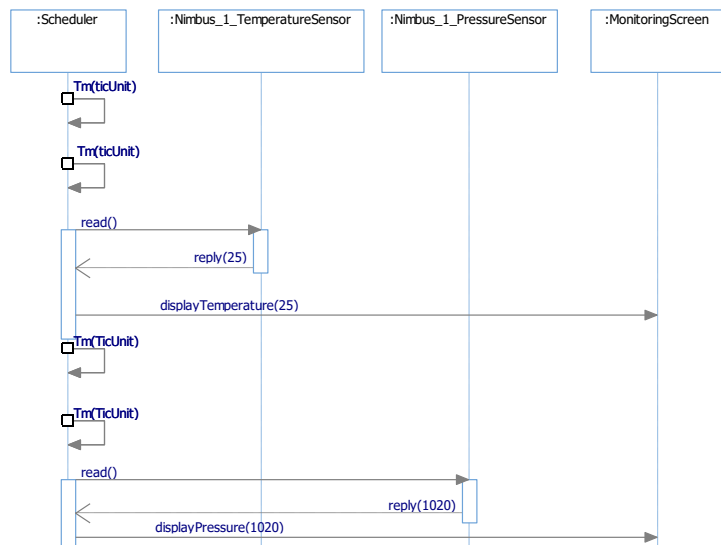


State



פונקציה	סוג	משמעות	פרמטרים	דוגמאות
stream	יצירה	יצירת Stream חדש מאוסף		list.stream()
parallelStream	יצירה	יצירת Stream מקבילי מאוסף		list.parallelStream()
Stream.of	יצירה	יצירת Stream באופן סטטי	רשימת ערכים	Stream.of("a", "b", "c")
Arrays.stream	יצירה	יצירת Stream ממערך	מערך	Arrays.stream(array)
Files.lines	יצירה	יצירת Stream משורות קובץ	מסלול לקובץ	Files.lines(Paths.get(path))
Stream.generate	יצירה	יצירת Stream אינסופי	Producer	IntStream.generate(()->random.nextInt(100))
IntStream.range	יצירה	יצירת Stream סדרת מספרים עוקבים	from, to	IntStream.range(8,15)
Stream.iterate	יצירה	יצירת Stream אינסופי בהינתן חוקיות	ערך התחלתי ו Function	IntStream.iterate(8, i->i+3)
filter	ביניים	סינון Stream	Predicate	filter(element -> element.contains("d"))
sorted	ביניים	מיון Stream	Comparator	sorted() sorted(Comparator.reverseOrder()) sorted((c1,c2) -> c1.getX()-c2.getX()) sorted(Comparator.comparing(Circle::getY))
map, mapToInt	ביניים	המרת איברי Stream	Function	map(person -> person.getPhoneNumber())
flatMap	ביניים	המרת כל איבר ל Stream ו"שיטוח" התוצאה	Function המחזירה Stream	flatMap(line -> Stream.of(line.split(" ")))
skip, limit	ביניים	זריקת/שמירת האיברים הראשונים בלבד	מספר האיברים	skip(5), limit(4)
distinct	ביניים	זריקת כפילויות		distinct()
boxed	ביניים	המרת Stream פרימיטיבי (לדוגמא IntStream) ל Stream המחלקה העוטפת		boxed()
collect	סופית	ממיר Stream לאוסף	Collector	collect(Collectors.toList()) collect(Collectors.toCollection(TreeSet::new)) collect(Collectors.groupingBy(Person::getCity)) collect(groupingBy(identity(), counting()))
reduce	סופית	מסכם Stream לערך יחיד	ערך התחלתי ו Function	Reduce(0, (subtotal, e) -> subtotal +e)
sum, average	סופית	מחזיר את הסכום/הממוצע של Stream של טיפוס מספרי (לדוגמא IntStream)		sum() average().orElse(0)
min, max	סופית	מחזיר את האיבר המינימלי/מקסימלי	Comparator	min().orElse(Integer.MAX_VALUE) min(Integer::compare).orElse(Integer.MAX_VALUE)
count	סופית	סופר כמה איברים יש ב Stream		
any/all/noneMatch	סופית	בודק אם אחד/כל/אף אחד מאברי ה Stream מקיים תכונה מסויימת	Predicate	anyMatch(element -> element>3)

תחנת לניטור מזג אוויר עיצוב ראשוני:



בעיות עיצוב:

- Scheduler (מתזמן): הפרת SRP (לא אמור להיות אחראי לעדכון המסך), הפרת DIP (לא אמור להיות תלוי במסך ובחיישנים).
- בעיית OCP: כאשר נעבור ל Nimbus2 כל המערכת תצטרך להבנות מחדש בגלל החלפת החיישנים.

שיפורים:

- הוספת שכבת הפשטה עבור החיישנים. שימוש ב Template Method (תרשים 1)
 - יתרונות:
 - הפשטה. ניתן לעבור ל Nimbus2 בלי לפתוח את הקוד לשינויים
 - תבנית העיצוב חוסכת שכפול קוד ושומרת על הקוד המשותף אחיד.
- ניתוק המסך מהמתזמן וחיבורו לעדכונים של החיישנים באמצעות תבנית Observer (תרשים 2)
 - מאחר ש MonitoringScreen משמש כ Observer של מספר חיישנים, נצטרך להוסיף מתאם ייעודי לכל חישן. כל מתאם יממש את הפונקציה update בצורה המתאימה לחישן הנצפה.
 - יתרונות:
 - פתרון בעיית ה SRP של השעון
 - אין תלות ישירה בין החיישנים למסך (שמירה על DIP)
- שימוש בתבנית עיצוב (לא רשמית) AlarmClock-AlarmListener (תרשים 3)
 - תבנית מקבילה ל Observer. בניגוד ל Observer, השעון מעיר כל פעם לקוח אחד.
 - יתרון: אין תלות ישירה בין החיישנים למתזמן (שמירה על DIP)

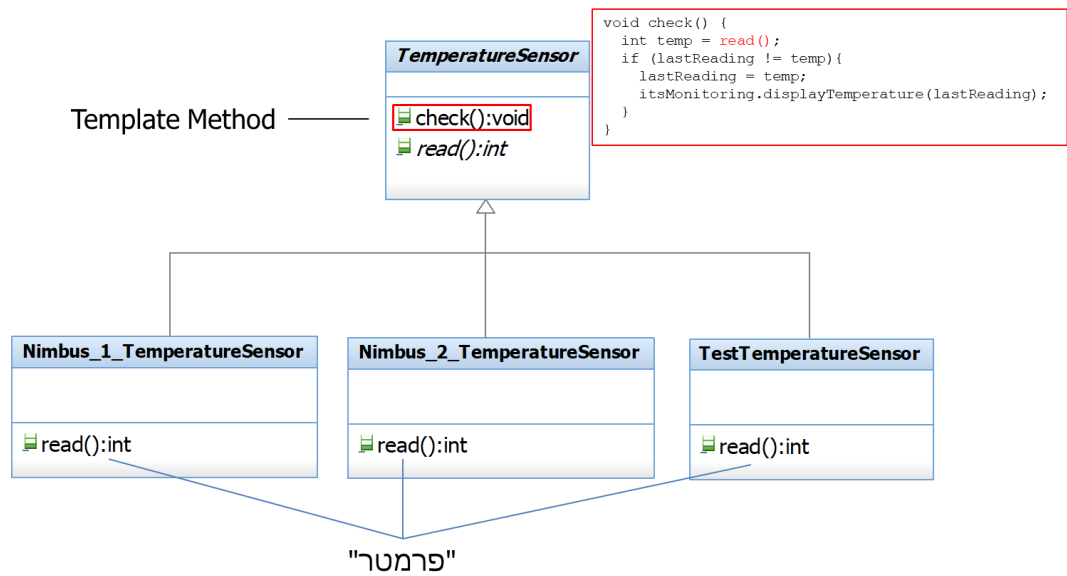


Figure 1 תבנית עיצוב Template Method

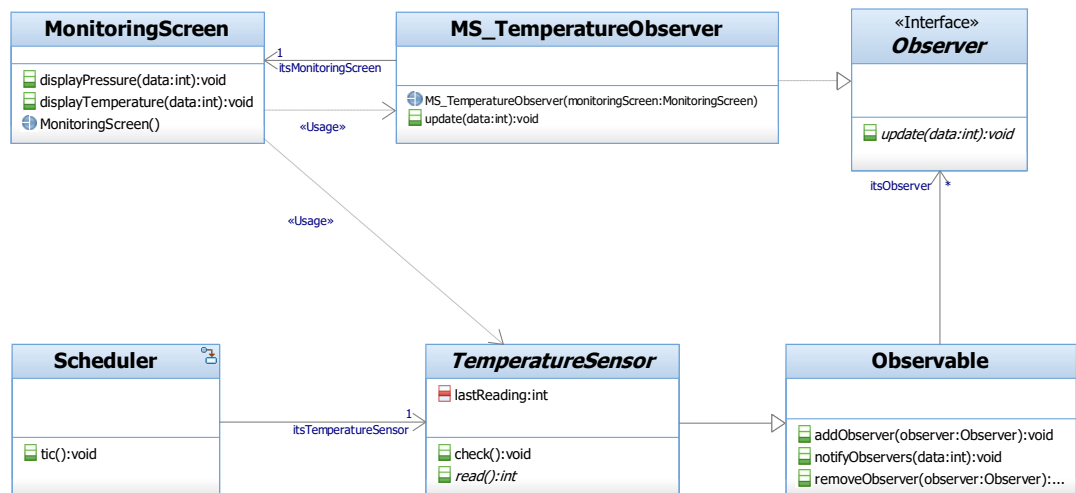


Figure 2 תבנית עיצוב Observer

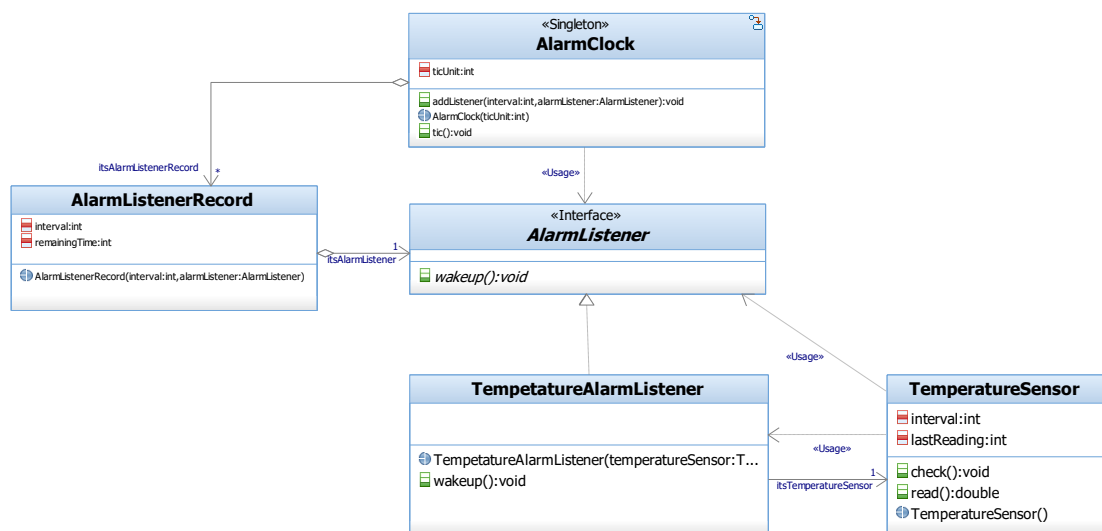


Figure 3 תבנית עיצוב AlarmClock-AlarmListener

הוספת חיישן למדידת מגמת לחץ האוויר

נהפוך את ה Observer לגנרי על מנת שיתמוך בסוגים שונים של עדכונים. החיפוש החדש יהיה Observer (יקבל עדכונים מסוג int מחיישן לחץ האוויר) ו Observable (יספק עדכונים מסוג Trend ל GUI, דרך מתאם ייעודי)

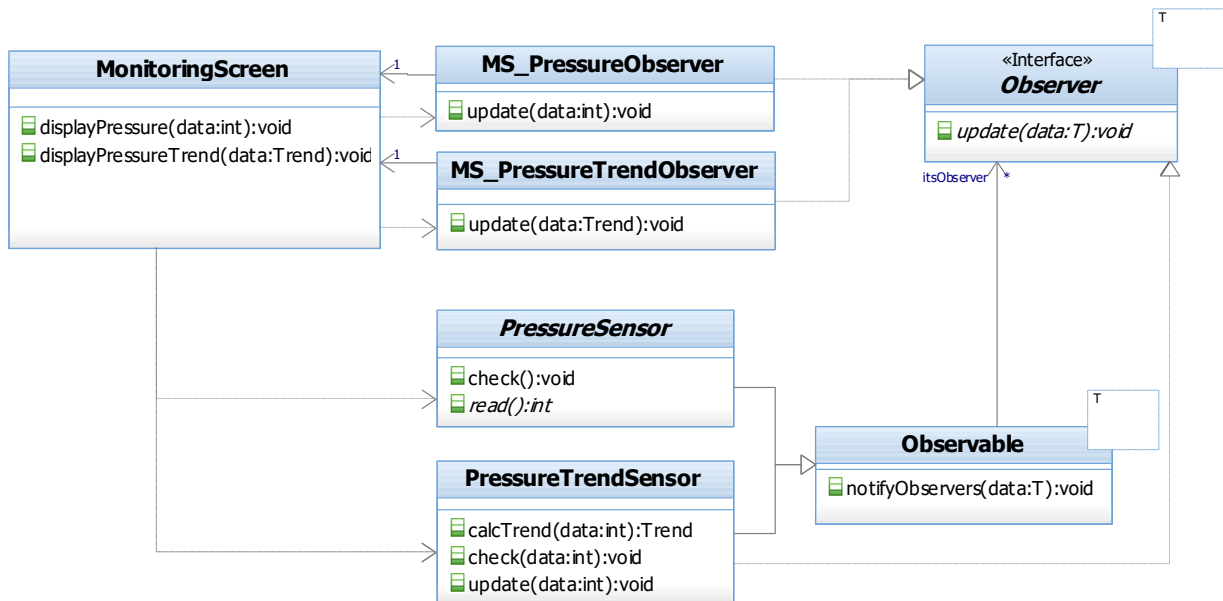


Figure 4 חיפוש שהוא Observer וגם Observable

תבנית עיצוב Bridge

בעיית עיצוב: החיפוש שמתחבר לחומרה (Nimbus1_0TemperatureSensor) תלוי בחיישן ברמת ההפשטה הגבוהה יותר שמתחבר לשעון ול Observer. עיצוב זה מייצר תלות שתגרום לחיישן החומרתי להפתח לשינויים בכל פעם שמשנים את רמת האפליקציה. על מנת להפריד בין שכבות הפשטה שונות בהיררכיה נשתמש בתבנית עיצוב Bridge. נוסיף ממשק בין השכבות. ממשק זה ימשש אותנו גם כ API שנוכל להעביר למהנדסי החומרה.

המחלקה TemperatureSensor כבר אינה מופשטת. הפונקציה read שלה תקרא ל read של TemperatureImp

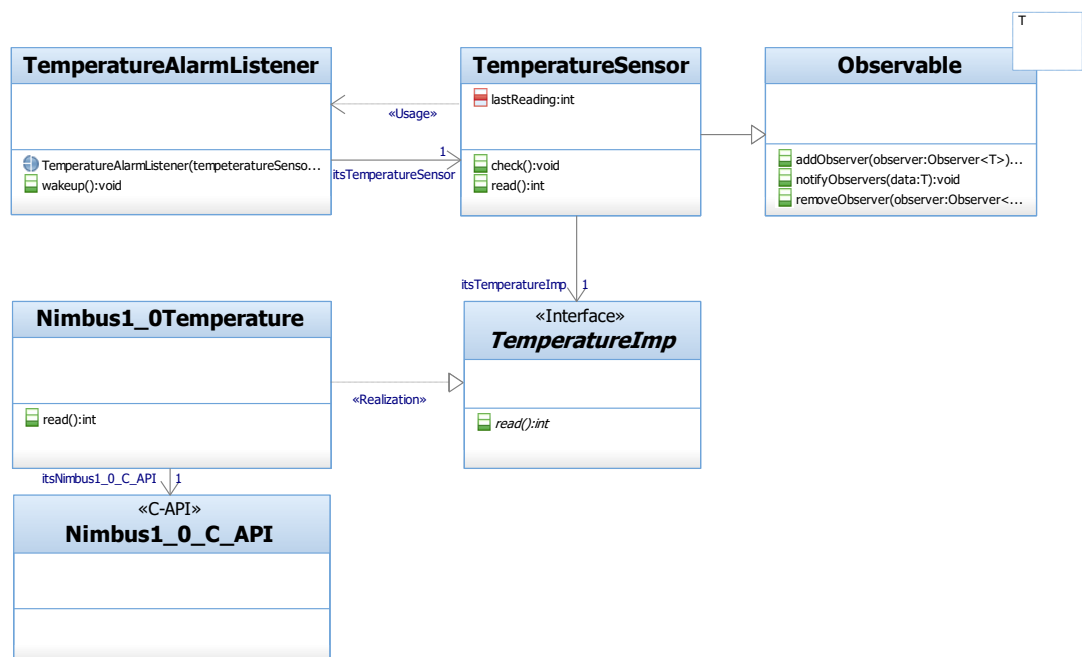


Figure 5 תבנית עיצוב Bridge עבור החיישן

שימוש ב Bridge עבור השעון

נפעיל את אותו ההגיון גם במקרה של השעון. הפעם השעון החומרתי מפעיל את השעון ברמת האפליקציה ולכן האחרון הוא זה שיממש את הממשק שנוסף. בנוסף נשתמש ברעיון של Observer-Observable כדי לאפשר לשעון החומרתי להעיר ClockListeners שונים (בעיצוב הנוכחי אין עוד שעונים, אך בעתיד נוכל להוסיף כאלה).

המחלקה AlarmClock היא כעת גם לקוחה של שירותי תזמון (ClockListener) שמונעת על ידי שכבת השעון החומרתי (על ידי הפונקציה tic) וגם משרתת לקוחות של שירותי תזמון (AlarmListener) שמניעה אותם (על ידי הפונקציה wakeup). יש כאן הקבלה לחיפוש המגמה שהוא גם Observer וגם Observable.

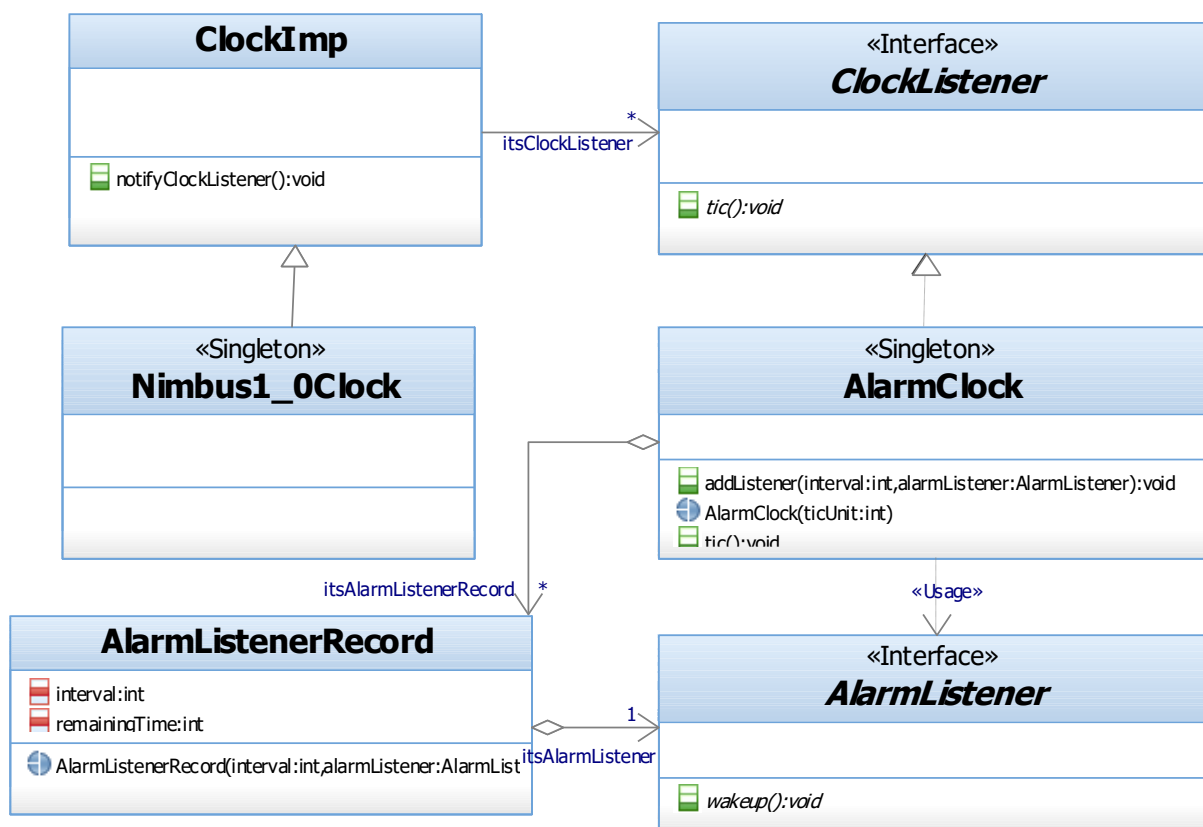


Figure 6 שילוב תבנית Bridge עם השעון

אתחול

תכנית ה Main תאתחל את רכיבי ה Gui (Log, MonitoringScreen) ואת ה WeatherMonitoringStation.

ה WeatherMonitoringStation תאתחל את השעון ואת החישנים ותעביר להם את הדרוש כדי שיתחברו למערכת. כמו כן מחלקה זו תכיל פונקציות שירות שמוסיפות Observers לחישנים השונים.

אפשרות א: ה WeatherMonitoringStation יאתחל בעצמו את המחלקות החומרתיות (Nimbus1/Nimbus2). הבנאי של החישנים/השעון ברמת האפליקציה יקבל את השעון ואת השכבה החומרתית ויתחבר אליה. חסרון: הפרת DIP. נצטרך לשנות את WeatherMonitoringStation בכל השורות שמתייחסות לחישנים ולשעון.

אפשרות ב: שימוש ב Abstract Factory. ה WeatherMonitoringStation יאתחל את המימוש של ה Factory. הבנאי של החישנים/שעון יקבל את ה Factory וידאג בעצמו לקבל מה Factory את המחלקה החומרתית. חסרון: עדיין נצטרך לשנות שורה אחת ב WeatherMonitoringStation כשנעבור לגרסה Nimbus2.

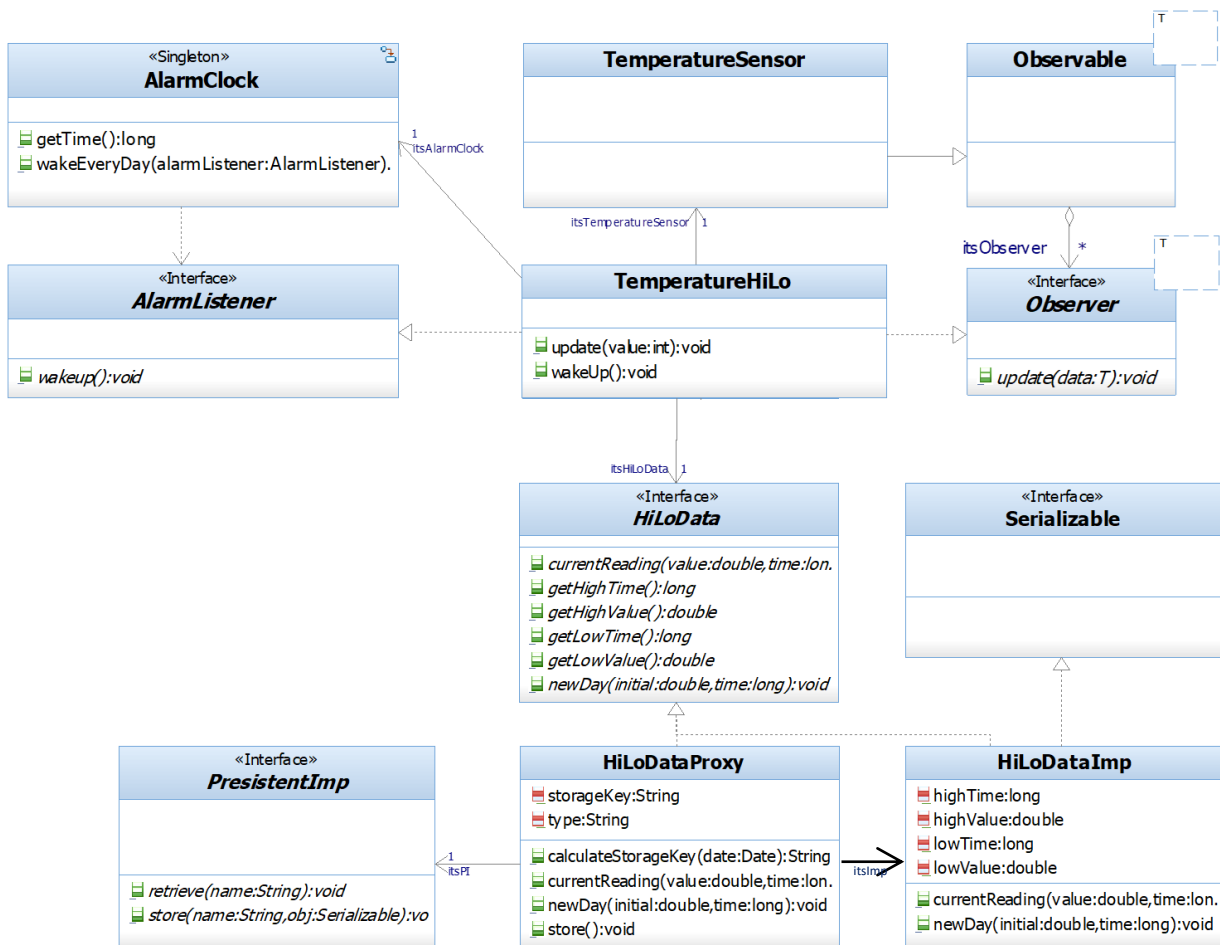
אפשרות ג: שימוש ב Reflection. ה WeatherMonitoringStation יקבל מחרוזת כפרמטר ויתחל את המימוש של ה Factory בעזרת המחלקה Class הגנרית והפונקציה `newInstance()` שלה. חסרון: אין בדיקת טיפוסים.

תבנית עיצוב Proxy

נרצה להוסיף תמיכה בחישוב ובשמירת ערכי הקיצון (מינימום ומקסימום) של הטמפרטורה.

שלב א': חיבור הרכיב לקליטים הנדרשים. הרכיב החדש מקבל עדכונים מהשעון (wakeEveryDay) ומהחישן (דרך מנגנון ה Observer) על כן נוסיף מחלקה TemperatureHilo שתהיה אחראית ליזום את פעולת הרכיב. בכל פעם שמתחלף יום נבדוק את הטמפרטורה ובכל פעם שמתעדכנת הטמפרטורה נבדוק מה השעה. שאר חלקי הרכיב יהיו ניתנים לשימוש חוזר עבור חישנים אחרים.

ברכיב החדש יש שני מנגנונים נפרדים: אלגוריתם החישוב של ערכי הקיצון ושמירת הנתונים לזיכרון הלא נדיף. על מנת להפריד את שתי האחריות הללו, נוסיף ביניהן שכבת Proxy. הפרוקסי ישמש כמתווך בין שכבת ה Persistence לשכבת ה Policy ובכל פעם שיפעילו את הרכיב החדש ה Proxy יפעיל ראשית את האלגוריתם ובמידת הצורך גם את השמירה לזיכרון הלא נדיף.

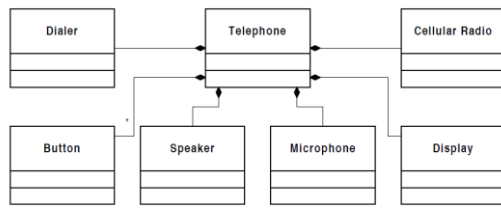


חלוקה לחבילות:

על פי החלוקה הראשונית נגדיר חבילה לכל גרסת Nimbus (כולל חבילת Test), חבילה עבור ה API (הממשקים) וחבילה נוספת שתכיל את כל שאר רכיבי האפליקציה (שעון, חישנים, UI והתחנה עצמה).

בחלוקה זו יש הפרת עקרון CCP. רכיבי ה UI אינם סגורים לאותו סוג שינויים כמו שאר הרכיבים בחבילת האפליקציה. על כן נפריד אותם לחבילה נפרדת. בחלוקה החדשה נוצרה הפרת ADP. יש תלות מעגלית בין התחנה לבין ה UI (ה WMS מייצר את רכיבי ה UI ורכיבי ה UI משתמשים ב WMS על מנת להתחבר לחישנים). על כן נוציא את האחריות לייצור ה UI למחלקה Main. נוצרה בעיית SDP (עבור חבילת ה UI, $I = \frac{1}{2}$). עבור חבילת ה WMS ($I = \frac{4}{7}$), על כן ננתק את התלות בין חבילת ה UI וחבילת ה WMS באמצעות שימוש בהיפוך התלות (הוספת ממשק שה WMS יממש ושיהיה בחבילה נפרדת שה UI יהיה תלוי בה).

מקרה בוחן טלפון סלולרי

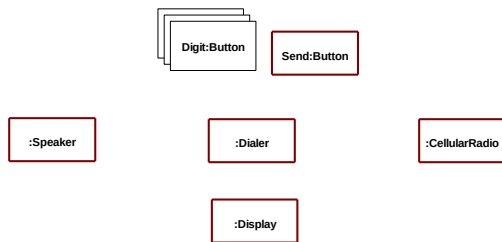


עיצוב ראשוני: נסיון לחקות את המבנה הפיזי של הטלפון כמכשיר שמורכב מחלקי משנה (מסך, מקשים וכו').

בעיות: הפרת SRP (הטלפון אחראי על הכל וכל השאר רכיבים "טפשים"). הטלפון מנהל את כל התקשורת בין הרכיבים, אחראי לאתחל אותם וכמובן תלוי בהם. הקשרים הדו כיווניים מובילים לצימוד הדוק של כל חלקי המערכת.

הפרת DIP (הטלפון תלוי ברכיבים ה"נמוכים"). כתוצאה מהפרות אלה מופר גם OCP (כל שינוי בכל אחד מחלקי המערכת מוביל לפתיחה לשינויים של כל שאר חלקי המערכת). לעולם לא נצליח לסגור אף מחלקה מפני שינויים.

נסיון שני: נתחיל בניתוח ההתנהגות. ההתנהגות היא מהות התוכנה ולא המבנה הפיזי של המכשיר הממודל.



נשים לב כי יש בתרשים לעיל ריבוי של כפתורי ספרות. בנוסף Digit ו Send הם מאותו טיפוס (Button).

נגזור מתרשים התקשורת את תרשים המחלקות. נשים לב כי בעיצוב החדש אין צורך במחלקת Telephone.

בעיות: הפרת DIP (Button תלוי ב Dialer), הפרת ISP (CellularRadio ו Dialer משתמשים ב Display, אך כל אחד צריך רק חלק מהממשק שלו).

עיצוב שלישי: נשתמש ב Dependency Inversion על מנת לפרק את התלות בין הכפתור לחיגן. מכיוון שלחייגן יש שתי דרכים שונות לממש את הפונקציה

buttonPress, נוסיף שני מתאמים ש"יתרגמו"

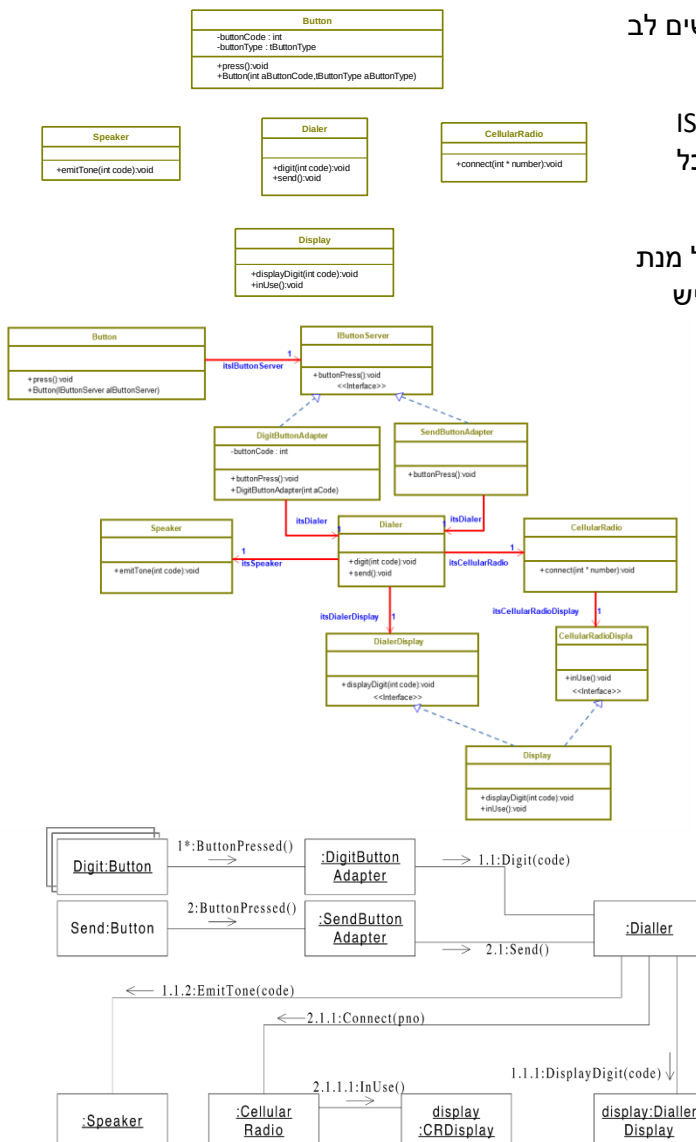
לחיצת כפתור ל"כפתור ספרה" ול"כפתור send".

נפצל את הממשק של Display לשני ממשקים, כך שמחלקות Dialer ו CellularRadio יחשפו רק לממשק הדרוש לכל אחד מהם.

נחזור לתרשים העיצוב המתאר את ההתנהגות ונעדכן אותו על פי תרשים המחלקות. נשים לב כי Send ו Digit הם מופעים של אותה המחלקה.

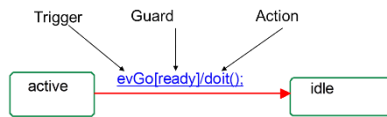
נשים לב כי האובייקט display מיוצג פעמיים בתרשים, פעם אחת כ CRDisplay ופעם שניה כ DialerDisplay. בפועל הוא יהיה מופע של Dialer, אך המשתמשים שלו מכירים רק את הממשקים שהוא מממש.

לסיכום: עיצוב הוא תהליך המשלב ניתוח התנהגות עם ניתוח מבנה המחלקות. שני המודלים (הדינמי והסטטי) משפיעים זה על זה. התכנה אמורה לשקף את התנהגות האובייקט ולא את המבנה הפיזי שלו.



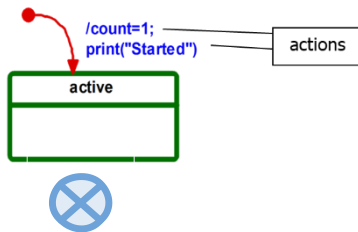
תרשימי מצבים

כללי: תרשימי מצבים מתאר את פעולתו של מחלקה אחת (מחלקה כזו תיקרא מחלקה תגובתית). מחלקה תגובתית נמצאת בכל רגע נתון במצב מסוים ויכולה לתקשר באמצעות שליחת וקבלת אירועים. שליחת אירוע הוא פעולה א-סינכרונית. המשמעות היא שהשולח אינו ממתין לערך החזרה או לסיום התגובה של המקבל. בנוסף המקבל אינו מחויב לטפל באירוע מייד עם קבלתו, ובמידה והוא באמצע טיפול באירוע אחר, האירוע החדש ייכנס לתור. אירועים בדרך כלל מיוצגים באמצעות שם מזהה. יש אירוע מיוחד tm המייצג timeout.



תרשימי מצבים בנויים ממצבים ומעברים ביניהם. מעבר מוגדר על ידי חץ ממצב מקור למצב יעד שעליו מוגדרים שלושה חלקים (אופציונליים):

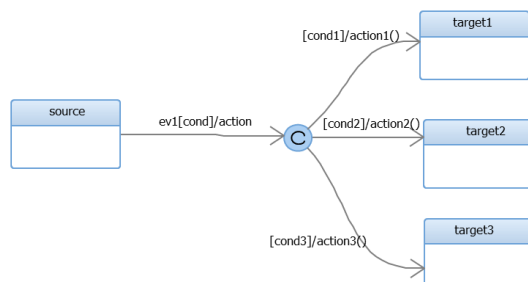
- Trigger: המעבר יתרחש רק אם האובייקט יקבל אירוע מסוג זה.
- Guard: המעבר יתרחש רק אם הביטוי יהיה שווה true.
- Action: במידה והמעבר מתרחש, יופעל הקוד.



מעבר ברירת מחדל: מסמן את המצב ההתחלתי של האובייקט (המצב בעת יצירת מופע). לא ניתן לציין במעבר ברירת מחדל trigger או guard.

מצב סופי: מצב בו האובייקט מפסיק את פעולתו.

למעשה שני המושגים 'מצב' ו'מעבר' שהוגדרו לעיל מספיקים כדי לממש את כל מגוון ההתנהגויות שלמדנו עליהן. יחד עם זאת, על מנת שתרשימי המצבים יהיה קל לכתובה, לקריאה ולשינוי, נרצה להוסיף שכלולים נוספים.



מקבץ תנאי: כאשר יש trigger משותף לכמה מעברים שיוצאים ממצב, ניתן למנוע את השכפול באמצעות מחבר שמאגד אותם יחד. אל המחבר נכנס אחד חץ עם ה trigger ויוצאים ממנו מספר חצים (ללא trigger).

סדר הפעולות במחבר תנאי: כאשר מתקבל ev1, נבדק cond (התנאי בחץ הנכנס). אם הוא true, נעבור על התנאים בחצים היוצאים. סדר המעבר אינו מוגדר על פי

התרשים ויש לוודא שההתנהגות תהיה עקבית ללא קשר לסדר. החץ הראשון שהתנאי שלו מתקיים יוביל לביצוע של action (הפעולה בחץ הנכנס), ואז לביצוע של actionX (הפעולה בחץ היוצא המתאים) ולמעבר למצב המתאים לחץ. אם אף אחד מהתנאים בחצים היוצאים לא מתקיים, הפעולה action לא תתבצע. המחבר חסך את השכפול של החלקים על החץ הנכנס.

תרשימי מכונת כביסה

לצורך פיתוח התרשימים השתמשנו בשכלולים נוספים

מעבר פנימי: ניתן להגדיר בתוך מצב מעבר שמייצג לולאה. בנוסף ניתן להגדיר במצב פעולה שתתבצע בכל פעם שנכנסים למצב או בכל פעם שיוצאים ממנו

היררכיה בין מצבים: כאשר יש מספר מצבים שיש להם אותם מעברים יוצאים, ניתן ליצור מצב המכיל אותם ובמקום לשכפל את המעבר, להוציא אותו מהמצב המכיל. אחד מבין המצבים המוכללים צריך להיות מוגדר כמצב ברירת מחדל

